# VéloVille

# Fiabiliser les données et suivre l’activité des vélos

3 ressources : stations, trajets et incidents

2 profils : exploitant et direction

# Architecture et périmètre livré

## Streamlit

4 écrans
Formulaires et tableaux

## FastAPI

JWT et contrôle du rôle
Services et contrôles

## PostgreSQL

4 tables, CHECK et FK
Données et agrégations

CSV : audit, fiabilisation, puis import vers PostgreSQL

**3 indicateurs : trajets par jour, stations actives, incidents et délai moyen**

Livré : import, API et front.
Hors périmètre : météo et anticipation.

# L’audit et l’import des données

```
def normaliser_statut(statut):
    if statut is None:
        raise ValueError("Statut absent")
    statut_normalise = statut.strip().lower().replace(" ", "_")
    if statut_normalise not in ["ouvert", "en_cours", "resolu"]:
        raise ValueError("Statut non reconnu")
    return statut_normalise
```

## Normalisation, rejet, doublon ou avertissement

Journal : 192 messages, dont 19 rejets et 12 doublons écartés

Après import : 120 stations, 2 002 trajets, 177 incidents

# La connexion et les droits

Exploitant : lecture et modification

Direction : lecture

401 : authentification absente ou invalide

403 : rôle insuffisant

**Succès exploitant : 200, statut en_cours**

```
Depends(exiger_role("exploitant"))
```

**Réponse attendue avec direction**

```
PUT /incidents/I2026-0065/statut
```

**403**

```
{"detail":
 "Acces reserve au role exploitant"}
```

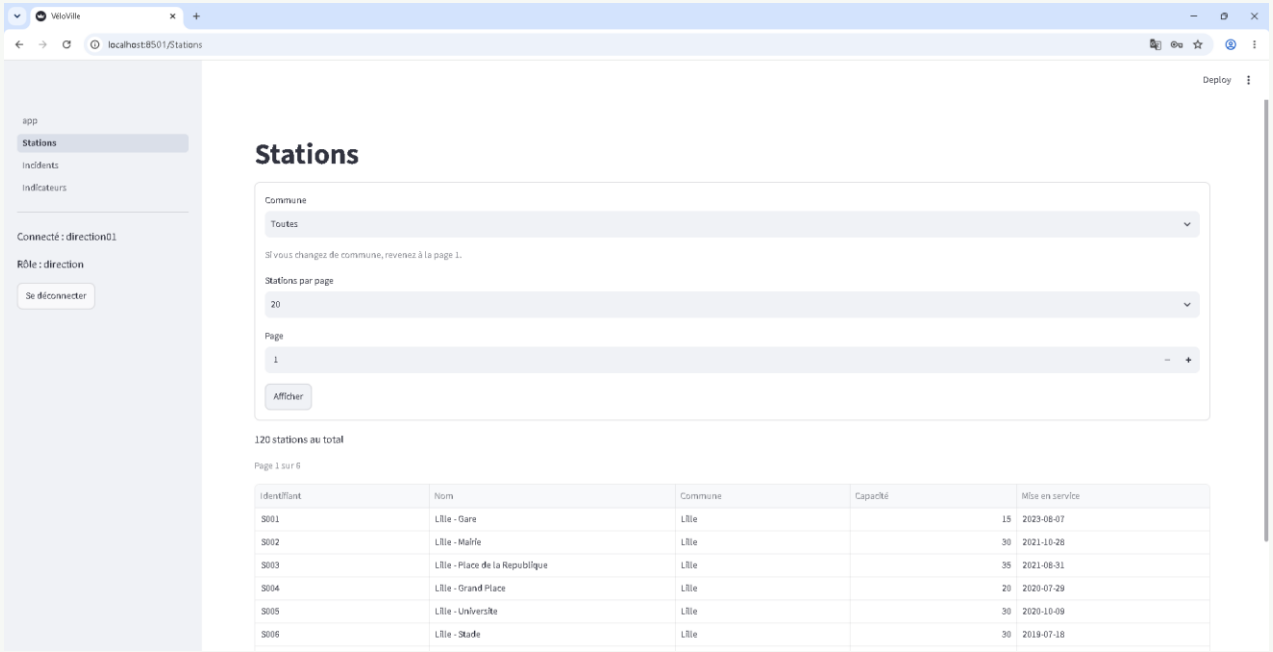
# Les quatre écrans Streamlit

Connexion : mot de passe masqué, session

Stations : commune et pagination

Incidents : filtre et suivi du statut

Indicateurs : 3 métriques, 3 tableaux




**Chargement, résultat vide et erreur : des messages en français**

# Un indicateur contrôlé avec SQL

## Même période : du 24 au 30 août 2026

```
SELECT CAST(depart_horodatage AS DATE) AS jour,
       COUNT(*) AS nb_trajets
FROM trajet
WHERE CAST(depart_horodatage AS DATE)
      BETWEEN '2026-08-24'
          AND '2026-08-30'
GROUP BY jour
ORDER BY jour;
```

| | jour<br>date | nb_trajets<br>bigint |
|---|---|---|
| 1 | 2026-08-24 | 329 |
| 2 | 2026-08-25 | 311 |
| 3 | 2026-08-26 | 306 |
| 4 | 2026-08-27 | 300 |
| 5 | 2026-08-28 | 315 |
| 6 | 2026-08-29 | 219 |
| 7 | 2026-08-30 | 222 |

## Référence : 329 le 24 août, 2 002 sur la période

# Les tests et leur couverture

**95 %**

Couverture globale de référence

Incidents : 57 %
Transitions et écritures à mieux couvrir

## Tests unitaires

Règles pures et réponses simulées

## Tests d’intégration

Routes HTTP et agrégats PostgreSQL

**Minimum demandé : 60 %**

```
python -m pytest --cov=api/services --cov-report=term-missing
```

# Les choix techniques et la qualité

## Organisation du code

Routeurs et dépendances
Services et modèles SQLAlchemy
Client HTTP centralisé

## Écoconception

Pagination : 20 par défaut, max. 100
Cache des communes : 1 heure
Actualisation à la demande
Agrégations SQL

## Tests complémentaires

Unitaires : règles et simulations
Intégration : API et PostgreSQL

## Sécurité

bcrypt, JWT signé et expiration
Rôle vérifié côté API, .env local

# La recette et les corrections

| Passage | OK | KO |
|---|---|---|
| Recette initiale | 8 | 2 |
| Après corrections et rejeux | 10 | 0 |

## ANO-001 : texte du refus 403

## ANO-002 : texte du 404 Station introuvable

Fiche, correction, test de non-régression et rejeu

# Les versions et la traçabilité

## Branches de travail fusionnées dans main

CHANGELOG et tags jusqu’à v0.0.9

## Dernier commit : 623e57b6c7bbb5d94ea184dfb9c9aca0e8f5bb91

Actualiser les documents

```
git status
git log -1 --oneline
```

## Pour enregistrer et envoyer

git add → git commit –m « commentaire » → git push

# Les limites et les axes d’amélioration

Compléter les tests automatisés des transitions d’incidents.

Utiliser une base dédiée aux tests.

Réinitialiser automatiquement la pagination après un changement de filtre.

Vérifier l’installation sur un autre poste.

**Priorité : une maintenance fiable et des résultats reproductibles**